

Dokumentacja Projektowa

SQL2Pandas – Konwerter SQL do operacji biblioteki Pandas

Zespół

Mikołaj Mydel, Michał Pastuszczyk

1 Cel Projektu

Program **SQL2Pandas** to zautomatyzowany kompilator (translator). Jego głównym zadaniem jest konwersja zapytań ustrukturyzowanego języka zapytań (SQL) na równoważne operacje transformacji danych w języku Python z wykorzystaniem funkcjonalności biblioteki **pandas**. Narzędziem odpowiedzialnym za generowanie parsera jest biblioteka **Lark**.

2 Architektura Rozwiązania

Proces translacji realizowany jest etapowo, od analizy surowego tekstu do wygenerowania kodu w systemie docelowym. W projekcie podzielono ten proces na trzy odpowiednio delegowane moduły:

1. **Analiza leksykalna i składniowa:** Wykonywana przy użyciu algorytmu Earley’a, operującego na regułach bezkontekstowych (mechanizm silnika biblioteki Lark).
2. **Abstrakcyjne Drzewo Składniowe (AST):** Konwersja wejściowego drzewa parsowania na uporządkowany zbiór obiektowych klas `dataclass`, odzwierciedlających reprezentację wykonywalną.
3. **Generowanie kodu docelowego:** Tłumaczenie i rekurencyjne przeglądanie struktur obiektu AST w celu zbudowania rdzenia Pythonowej syntaktyki.

2.1 Definicja Gramatyki (`sql.lark`)

Silnik parsujący opiera się o gramatykę zadeklarowaną w pliku `sql.lark`. Analizator parsuje pełny zbiór zdefiniowanych leksemów:

- Słowa kluczowe niewrażliwe na wielkość liter (*case-insensitive*): `SELECT`, `DISTINCT`, `INSERT`, `INTO`, `VALUES`, `UPDATE`, `SET`, `DELETE`, `FROM`, `AS`, `JOIN`, `INNER`, `LEFT`, `RIGHT`, `FULL`, `ON`, `WHERE`, `AND`, `OR`, `NOT`, `IS`, `NULL`, `LIKE`, `IN`, `BETWEEN`, `GROUP BY`, `HAVING`, `ORDER BY`, `ASC`, `DESC`, `LIMIT`.
- Funkcje agregujące: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`.

- Operatory porównania i arytmetyczne: `=`, `!=`, `<>`, `>`, `<`, `>=`, `<=`, `+`, `-`, `*`, `/`.
- Typy literalów oraz identyfikatorów: `NAME` (zmienne/kolumny), `INTEGER` (liczby całkowite), `NUMBER` (liczby zmiennoprzecinkowe), `STRING` (łańcuchy znaków).

2.2 Węzły AST (`ast_nodes.py`)

Logika reprezentacji abstrakcyjnego drzewa składniowego opiera się na definicji wszystkich wspieranych struktur z wykorzystaniem wbudowanego modułu `dataclasses`. Składa się na nie pełna lista następujących klas:

- **Struktury instrukcji operacyjnych (`Stmt`):** `SelectStmt`, `InsertStmt`, `UpdateStmt`, `DeleteStmt`.
- **Obiekty objaśniające reguły warunkowe (`Condition`):** `Compare`, `IsNull`, `IsNotNull`, `LikeCond`, `NotLike`, `InCond`, `NotIn`, `Between`, `NotBetween`, `AndCond`, `OrCond`, `NotCond`.
- **Wyrażenia operacyjne i referencje (`Expr`):** `ColExpr`, `QualifiedCol`, `NumberLiteral`, `StringLiteral`, `BinOp`, `Neg`, `AggStar`, `AggExpr`.
- **Pomocnicze struktury danych i klauzule zapytań:** `Having`, `SelectItem`, `TableRef`, `JoinClause`, `OrderItem`.

2.3 Konwersja na obiekty abstrakcyjne (`tree_to_ast.py`)

Za weryfikację i logiczne przekształcenie surowych struktur (węzły `Tree` od parsera `Lark`) do wyżej wymienionych węzłów klasowych odpowiada klasa `TreeToAST` (dziedzicząca z `Transformer`). Posiada ona absolutne mapowanie wszystkich potrzebnych struktur języka poprzez następujące zadeklarowane metody transformacji:

- **Metody wyrażeń węzłowych:** `col`, `qualified_col`, `number`, `string`, `add`, `sub`, `mul`, `div`, `neg`, `agg_star`, `agg_expr`.
- **Metody budowania warunków logicznych:** `compare`, `eq`, `neq`, `lt`, `gt`, `lte`, `gte`, `is_null`, `is_not_null`, `like_cond`, `not_like`, `in_cond`, `not_in`, `between_cond`, `not_between`, `and_cond`, `or_cond`, `not_cond`, `value_list`.
- **Metody składowe klauzul `select` i relacyjnych:** `select_item`, `star`, `select_list`, `table_ref`, `join_clause`, `inner_join`, `left_join`, `right_join`, `full_join`, `where_clause`, `groupby_clause`, `having_clause`.

2.4 Kompilacja i generator pandas (`ast_to_pandas.py`)

Silnik transpilatora oparty jest na klasie `ASTToPandas`, która poprzez metody nawigujące (takie jak główne `gen`, `gen_expr`, `gen_condition`) precyzyjnie generuje łańcuchy uchodzące za format wykonywalny ramki danych biblioteki `pandas` (`DataFrame`). Generowane są wszystkie poniższe zachowania:

- Budowanie podstawowych transformacji instrukcji: `_gen_insert`, `_gen_update`, `_gen_delete`, `_gen_select`, `_gen_select_body`, `_gen_simple`, `_gen_groupby`.

- Tłumaczenie wszystkich warunków logicznych (prywatne metody ewaluacji masek na kolumnach typu boolean): `_cond_compare`, `_cond_is_null`, `_cond_is_not_null`, `_cond_like`, `_cond_not_like`, `_cond_in`, `_cond_not_in`, `_cond_between`, `_cond_not_between`, `_cond_and`, `_cond_or`, `_cond_not`.
- Wyliczanie wartości, literałów oraz wywołań poszczególnych zmiennych w kolumnach (prywatne metody operacji na stałych i arytmetyki): `_expr_col`, `_expr_qualified_col`, `_expr_number`, `_expr_string`, `_expr_binop`, `_expr_neg`, `_expr_agg_star`, `_expr_agg_expr`.
- Narzędzia formatujące i sortujące obsługujące specyfikę API pandas: `_apply_orderby`, `_extract_join_keys`, `_col_name`, `_agg_func`, `_gen_having_condition`, `_having_expr`.

3 Instrukcja Uruchomienia (`main.py`)

Obsługiwane są wyłącznie dwa tryby interakcji (zaimplementowane z poziomu głównego skryptu):

1. **Tryb procesowania wsadowego (funkcja `main()`):** Kompiluje sekwencję testowej listy zawierającej zapytania (stała operacyjna `QUERIES`), formatując wypisanie SQL oraz odpowiadającą i uruchamialną instrukcję `pandas`.
2. **Strumień interaktywny (funkcja `repl()`):** Uruchamia ciągle nasłuchiwanie zapytania w terminalu i jego kompilację tuż po zatwierdzeniu. Do zaprzestania działania pętli należy użyć jednoznakowej komendy wyjściowej `q`.

4 Wymagania systemowe

Zarządzanie środowiskiem realizowane jest przez narzędzie `uv`. Definicja pakietu opisana jest w pliku konfiguracyjnym `pyproject.toml`. Zgodnie ze specyfikacją oprogramowanie wymaga:

- Python ≥ 3.14
- `lark` $\geq 1.3.1$ (narzędzie do budowy maszyn i parserów składniowych)
- `pandas` $\geq 3.0.2$ (biblioteka do generowania oczekiwanych metod operacyjnych na wynikach)